HDOC Day-62

Merging Two Sorted Arrays Using Pointers in C

Problem Statement

Read two sorted variable-length integer arrays and merge them into a third array in ascending order using a function named merge_sorted_arrays(). Pass the arrays using pointers, preserve duplicates, and display the merged values separated by spaces.

Algorithm

1. Start.

2. Read the positive size n1 of the first array.

3. Declare a variable-length array arr1[n1] and read its elements in ascending order.

4. Read the positive size n2 of the second array.

5. Declare a variable-length array arr2[n2] and read its elements in ascending order.

6. Call merge_sorted_arrays(arr1, n1, arr2, n2), passing array addresses.

7. Inside the function, declare merged[n1+n2] and initialize i=j=k=0.

8. While both arrays have remaining elements, copy the smaller current element to merged[k], advancing the corresponding index and k. Use <= to preserve duplicates.

9. Copy any remaining elements of arr1 and then arr2 into merged.

10. Display merged elements separated by spaces.

11. Stop.

Test Case Table

Test case	Array 1: size; elements	Array 2: size; elements	Expected output
1	4; 1 3 5 7	4; 2 4 6 8	1 2 3 4 5 6 7 8
2	4; 1 2 2 4	3; 2 3 5	1 2 2 2 3 4 5
3	3; 1 4 7	5; 2 3 5 6 8	1 2 3 4 5 6 7 8
4	1; 5	1; 2	2 5
5	3; -5 -2 0	3; -3 0 4	-5 -3 -2 0 0 4
6	3; 1 1 1	2; 1 1	1 1 1 1 1
7	3; 2 4 6	2; 7 9	2 4 6 7 9
8	3; 8 9 10	2; 1 2	1 2 8 9 10
9	0 (invalid size)	Not read	Invalid first array size.
10	3; 1 3 2 (unsorted)	Not read	First array must be sorted in ascending order.

Note: The array size is entered before its elements. Test cases 9 and 10 check input validation. Duplicate values are retained.

C Program

```
#include <stdio.h>

void merge_sorted_arrays(const int *arr1, int n1, const int *arr2, int n2)
{
    int merged[n1 + n2];
    int i = 0, j = 0, k = 0;
    while (i < n1 && j < n2) {
        if (*(arr1 + i) <= *(arr2 + j))
            merged[k++] = *(arr1 + i++);
        else
            merged[k++] = *(arr2 + j++);
    }
    while (i < n1) merged[k++] = *(arr1 + i++);
    while (j < n2) merged[k++] = *(arr2 + j++);
    printf("Merged array: ");
    for (i = 0; i < k; i++)
        printf("%d%s", merged[i], i == k - 1 ? "\n" : " ");
}

int main(void)
{
    int n1, n2, i;
    printf("Enter size of first array: ");
    if (scanf("%d", &n1) != 1 || n1 <= 0 || n1 > 10000) {
        printf("Invalid first array size.\n"); return 1;
    }
    int arr1[n1];
    printf("Enter %d sorted elements of first array:\n", n1);
    for (i = 0; i < n1; i++) {
        if (scanf("%d", arr1 + i) != 1) {
            printf("Invalid element.\n"); return 1;
        }
        if (i > 0 && arr1[i] < arr1[i-1]) {
            printf("First array must be sorted in ascending order.\n"); return 1;
        }
    }
    printf("Enter size of second array: ");
    if (scanf("%d", &n2) != 1 || n2 <= 0 || n2 > 10000) {
        printf("Invalid second array size.\n"); return 1;
    }
    int arr2[n2];
    printf("Enter %d sorted elements of second array:\n", n2);
    for (i = 0; i < n2; i++) {
        if (scanf("%d", arr2 + i) != 1) {
            printf("Invalid element.\n"); return 1;
        }
        if (i > 0 && arr2[i] < arr2[i-1]) {
            printf("Second array must be sorted in ascending order.\n"); return 1;
        }
    }
    merge_sorted_arrays(arr1, n1, arr2, n2);
    return 0;
}
```

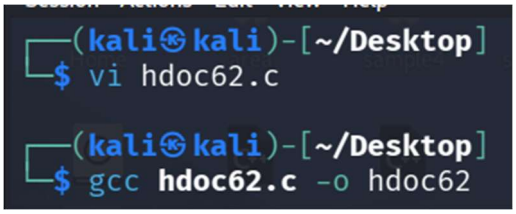
Compilation and Execution

Save the source as hdoc62.c, then use these commands in Kali Linux:

```
gcc hdoc62.c -o hdoc62
```

```
./hdoc62
```

Compilation and Execution Screenshot

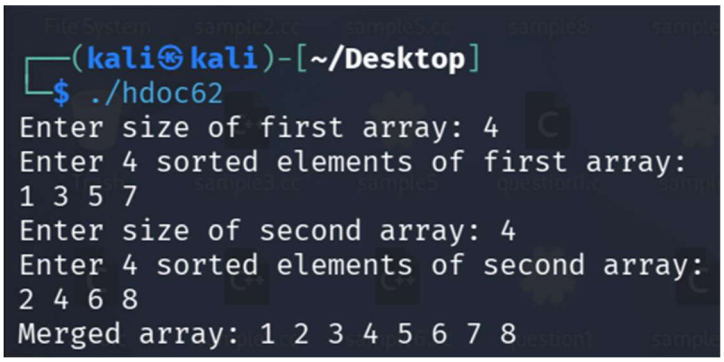


Test Case 1

Array 1: 4; 1 3 5 7

Array 2: 4; 2 4 6 8

Expected result: 1 2 3 4 5 6 7 8

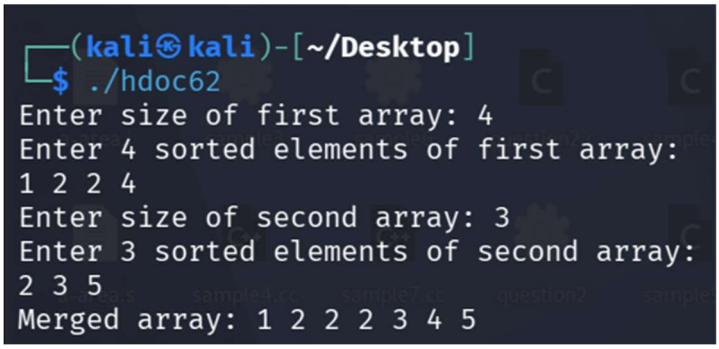


Test Case 2

Array 1: 4; 1 2 2 4

Array 2: 3; 2 3 5

Expected result: 1 2 2 2 3 4 5

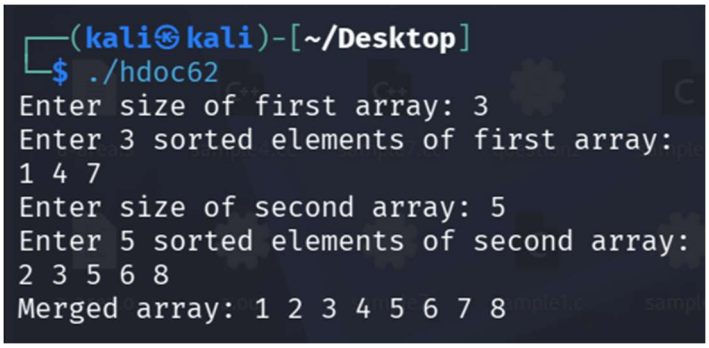


Test Case 3

Array 1: 3; 1 4 7

Array 2: 5; 2 3 5 6 8

Expected result: 1 2 3 4 5 6 7 8

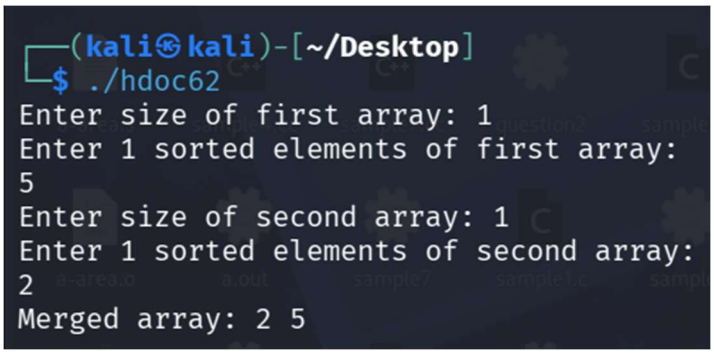


Test Case 4

Array 1: 1; 5

Array 2: 1; 2

Expected result: 2 5



Test Case 5

Array 1: 3; -5 -2 0

Array 2: 3; -3 0 4

Expected result: -5 -3 -2 0 0 4

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc62
Enter size of first array: 3
Enter 3 sorted elements of first array:
-5 -2 0
Enter size of second array: 3
Enter 3 sorted elements of second array:
-3 0 4
Merged array: -5 -3 -2 0 0 4
```

Test Case 6

Array 1: 3; 1 1 1

Array 2: 2; 1 1

Expected result: 1 1 1 1 1

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc62
Enter size of first array: 3
Enter 3 sorted elements of first array:
1 1 1
Enter size of second array: 2
Enter 2 sorted elements of second array:
1 1
Merged array: 1 1 1 1 1
```

Test Case 7

Array 1: 3; 2 4 6

Array 2: 2; 7 9

Expected result: 2 4 6 7 9

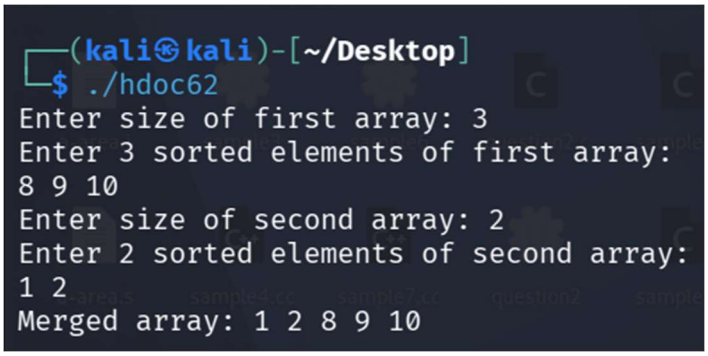
```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc62
Enter size of first array: 3
Enter 3 sorted elements of first array:
2 4 6
Enter size of second array: 2
Enter 2 sorted elements of second array:
7 9
Merged array: 2 4 6 7 9
```

Test Case 8

Array 1: 3; 8 9 10

Array 2: 2; 1 2

Expected result: 1 2 8 9 10

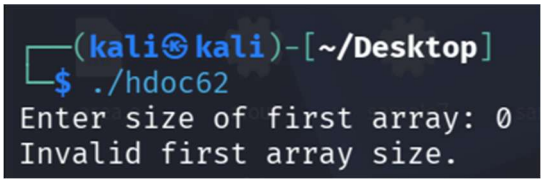


Test Case 9

Array 1: 0 (invalid size)

Array 2: Not read

Expected result: Invalid first array size.



Test Case 10

Array 1: 3; 1 3 2 (unsorted)

Array 2: Not read

Expected result: First array must be sorted in ascending order.

